Відокремлений структурний підрозділ
«Економіко-технологічний фаховий коледж
Херсонського національного технічного університету»

ПРАКТИЧНА РОБОТА № 1

з дисципліни «Проєктний практикум»

Тема: «Ініціювання командного ІТ-проєкту»

Проєкт «EasyStock»

Виконав:
студент групи ЗІПЗс-24
Бондаренко Сергій

Перевірив:
Ігор Косенко

Хмельницький 2026

Мета роботи та статут проєкту

Я обґрунтував вибір EasyStock, визначив користувачів, SMART мету, критерії успіху, межі MVP та порядок індивідуальної роботи. Підготовлений застосунок реалізує наскрізний складський сценарій; автоматизовані перевірки основної логіки пройдено.

Статут проєкту наведено на цій і наступній сторінках. Єдиний виконавець і відповідальний за всі артефакти — Бондаренко Сергій, група ЗІПЗс.

Проблема та цінність

Цільовий контекст — невеликий магазин або склад, у якому комірник приймає і списує товар, а менеджер перевіряє наявність перед продажем. Розрізнені таблиці та ручні підрахунки можуть створювати неузгоджені залишки, зайві уточнення і складність встановлення автора зміни. Це робоча гіпотеза навчального проєкту, а не результат вимірювання конкретного підприємства.

EasyStock має надати єдиний каталог, пошук за назвою або SKU, контроль кількості й журнал операцій. Користувач бачить результат у тому самому процесі, а залишок змінюється разом із записом історії.

SMART мета

До 05.10.2026 підготувати працездатний MVP EasyStock, у якому авторизований комірник може знайти товар, провести надходження, виконати допустиме списання і перевірити історію; надмірне списання має відхилятися без зміни даних. Технічне приймання — успішна production збірка та проходження всіх перевірок критичних правил у pytest.

Метрика	Початковий стан	Ціль і перевірка	Стан
Час пошуку товару людиною	Не виміряно	≤30 с; 5 повторів пошуку за SKU у свіжій демобазі	Потрібен людський прогін
Цілісність складських операцій	До тестування немає доказу	0 порушень у позитивних, негативних і конкурентних сценаріях	Тести пройдено
Час відповіді API	До тестування не виміряно	p95 ≤2 с; 30 запитів за 500 товарів	У TESTING.md: 0,0037 с

Локальний p95 вимірювався через Flask test client. Він не вимірює затримку мережі чи час дій користувача. На перевіреній версії GitHub повторно пройдено 39 тестів і production збірку.

Статут проєкту та межі продукту

Цільова група і зацікавлені сторони

Сторона	Потреба	Вплив	Порядок залучення
Комірник	Коректно проводити операції та бачити результат	Високий	Змодельована сесія і сценарій перевірки
Менеджер	Швидко знаходити товар і актуальний залишок	Середній	Змодельована сесія; майбутній usability прогін
Адміністратор	Керувати каталогом та доступом	Високий	Самоперевірка прав і довідників
Власник малого бізнесу	Прозорий облік і контроль дефіциту	Високий	Перевірка продуктових метрик у майбутньому
Викладач	Оцінити артефакти й обґрунтування рішень	Високий	Посилання на GitHub і захист
Бондаренко Сергій	Реалізувати та пояснити проєкт	Високий	Усі ролі виконую самостійно

Ядро MVP

Вхід → каталог і пошук → картка товару → надходження або списання → оновлений залишок → історія → вихід. Мінімальний каталог підтримує створення товару та базові довідники. Доступ контролюється сервером для трьох ролей застосунку.

Поза початковою межею MVP

CSV обмін, сканування штрихкодів, розширена аналітика, мобільний застосунок і зовнішні повідомлення не є умовою приймання ядра. У поточній версії додатково реалізовано редагування товарів, низькі залишки та керування користувачами; це розширення початкового плану.

Обмеження і критерії готовності

Навчальний локальний вебзастосунок; SQLite на постійному диску; кількість товару — ціле число; один розробник. Критерій технічної готовності: інсталяція за README, демонстраційний сценарій, 39 успішних тестів, збірка React і доступний GitHub.

Методичка передбачає командну роботу. Проєкт фактично виконано мною одноосібно; індивідуальний формат і заміну взаємної перевірки самоперевіркою потрібно погодити з викладачем. Таке погодження у цьому звіті не заявляю.

Вибір проблеми та відповідальність

Я порівняв три навчальні варіанти. Оцінки є моєю експертною оцінкою за шкалою 1–5, а не статистикою користувачів. Вищий бал означає кращу придатність; для ризику 5 означає нижчий ризик.

Варіант	Користь	Перевірка	Виконання	Ризик	Сума
FixFlow облік ремонтів	4	3	4	3	14
StudyHub навчальна платформа	4	3	4	3	14
EasyStock складський облік	5	4	5	4	18

Я обрав EasyStock, тому що правила залишку можна перевірити однозначно, а навчальні дані не потребують персональних або виробничих записів. Доступ до реальних працівників не підтверджено, тому моделювання придатне для формування вимог, але не доводить ринкову користь.

Індивідуальна матриця відповідальності

R означає виконання, A — відповідальність за результат. Оскільки роботу виконую один, в усіх внутрішніх завданнях R і A поєдную я. Викладач є стороною навчального оцінювання, а не учасником розробки.

Артефакт або дія	Поєднана роль	R і A
Статут, цілі, пріоритети	Автор ідеї та керівник проєкту	Бондаренко Сергій
Персони, backlog, критерії	Бізнес аналітик	Бондаренко Сергій
Архітектура, модель даних, ADR	Проєктувальник	Бондаренко Сергій
API, React, база даних	Розробник	Бондаренко Сергій
Тести і самоперевірка	Тестувальник	Бондаренко Сергій
GitHub, звіти, захист	Автор документації	Бондаренко Сергій

Мій особистий внесок підтверджується кодом і документацією у репозиторії, результатами тестів та поясненням рішень на захисті. Наступні зміни оформлення зафіксовано окремими змістовними комітами у main.

Правила роботи та план завершення

Я сформулював індивідуальні робочі правила замість угоди між кількома учасниками. Вони регулюють мою роботу та майбутню комунікацію з викладачем; підписаної командної угоди немає.

Правило	Як застосовую
Єдине джерело версії	Код, інструкції та артефакти зберігаю у GitHub; у звітах вказую версію.
Фіксація завдань	Кожне завдання повинне мати результат, статус, критерій завершення і виконавця Бондаренко Сергій.
Комунікація	Планую відповідати на навчальні уточнення протягом 24 годин у робочі дні.
Зміна пріоритетів	Спочатку забезпечую цілісність MVP; рішення і мотив записую у backlog або ADR.
Перевірка змін	Для бізнес правил запускаю відповідні тести; для інтерфейсу перевіряю основний і помилковий сценарії.
Готовність зміни	Код працює, дані узгоджені, інструкція відповідає реалізації, результат зафіксований.

Завдання для завершення подання

Завдання	Критерій завершення	Статус	Виконавець
Перевірити застосунок	39 тестів і збірка PASS	Виконано	Бондаренко Сергій
Підготувати 3 звіти	Є вимоги, діаграми, прототип і GitHub	Виконано у цих звітах	Бондаренко Сергій
Очистити Git від БД і кешу	БД, WAL, SHM і рус не відстежуються	Виконано 06.10.2026	Бондаренко Сергій
Додати ignore і оновити README	Є .gitignore, актуальні посилання й автор	Виконано 06.10.2026	Бондаренко Сергій
Створити дошку GitHub	Є доступне викладачу посилання і задачі	Виконано 06.10.2026	Бондаренко Сергій
Перевірити браузери й зручність	Записані результати і час пошуку	До фінального приймання	Бондаренко Сергій

План реалізовано у публічній дошці GitHub Projects із 14 задачами, критеріями завершення та єдиним виконавцем chaek438 — Бондаренком Сергієм. Посилання на дошку наведено в матеріалах проєкту на GitHub. Дедлайн для невиконаних пунктів має визначити викладач.

Перевірка репозиторію GitHub

https://github.com/chaek438/EasyStock

Перевірений стан оформлення на 06.10.2026: репозиторій public, гілка main, commit 203354f2bc96, 63 відстежувані файли. GitHub доступний за посиланням без запрошення викладача. Public означає, що матеріали можуть переглядати й інші користувачі.

Перевірка	Фактичний результат	Оцінка
Структура	backend, frontend, database, docs, tests, README і стартові скрипти у корені	Коректна
Тести API і правил	39 passed, повторна перевірка клону, 25,43 с	Пройдено
Production збірка	Vite завершив збірку React без помилки	Пройдено
.gitignore	Додано правила для Python, Node.js, SQLite та локальних налаштувань	Виконано
Робочі дані і кеш	У поточній main немає БД, WAL, SHM та кешу Python	Виконано
Інструкції GitHub	README і Git.md містять автора, звіти та фактичні посилання	Оновлено
Історія main	Зміни оформлення й очищення зафіксовано змістовними комітами	Оновлено
Дошка GitHub Projects	Публічна дошка з 14 задачами, критеріями та одним виконавцем	Створено

Виконані виправлення оформлення

Додано .gitignore для віртуальних середовищ, node_modules, кешу Python, локальних SQLite баз, WAL, SHM та .env. Робочу БД і сім рус файлів прибрано з поточної main. Локальні дані створюються при демозапуску зі схеми й demo seed; попередні коміти залишаються в історії Git.

README містить «Виконав: Бондаренко Сергій», посилання на три звіти та публічну дошку. У docs/Git.md вказано фактичний репозиторій. Дошка пов’язана з EasyStock, а всі 14 карток призначено одному виконавцю.

Висновок перевірки: реалізація та структура працездатні, оформлення GitHub оновлено. Залишаються людська перевірка зручності, вимірювання часу пошуку, повний прогін у Chromium і Firefox та погодження індивідуального формату з викладачем.

Короткий захист і висновок

Текст виступу до трьох хвилин

Я, Бондаренко Сергій, самостійно розробив EasyStock — навчальний вебзастосунок складського обліку. Проєкт розв’язує задачу узгодження каталогу, залишку та історії операцій для невеликого магазину. Я обрав цю тему серед обліку ремонтів, навчальної платформи та складського обліку, оскільки складські правила можна перевірити точно.

Ядро MVP — вхід, пошук товару, надходження, списання й перегляд історії. У демонстраційній базі товар SSD-480 має 17 одиниць. Надходження 20 змінює залишок на 37; списання 40 відхиляється; списання 3 залишає 34. Автор і час успішної операції потрапляють до журналу. Менеджер може переглядати дані, але не проводити складські операції.

Я використав React, Flask, SQLAlchemy та SQLite. Залишок і журнал зберігаються однією транзакцією, а умовне SQL списання захищає від надмірного списання за одночасних запитів. Технічна перевірка підтвердила 39 тестів і production збірку. Я підготував вимоги, діаграми, модель даних і прототип, оновив GitHub та дошку задач. Людська перевірка зручності ще запланована.

Висновок

Я сформував узгоджену основу проєкту і перевірив його реалізоване ядро. Усі проєктні ролі та всі роботи виконав самостійно. Головний технічний ризик — цілісність залишків — перевірено тестами; час пошуку людиною та браузерна сумісність залишаються окремими критеріями приймання.

Використані матеріали

Методичні матеріали викладача «Ініціювання командного IT проєкту»: розгорнутий текст на 23 сторінках і презентація на 45 сторінках.

Код і документація проєкту: https://github.com/chaek438/EasyStock

Перевірений commit 8e8a046cee04

Результати технічних перевірок

Відповіді на контрольні запитання

1 Чому проблема придатна для практикуму

Є конкретні ролі, наскрізний процес, сутності та правила з вимірюваним результатом; демонстрація можлива на безпечних навчальних даних.

2 Що є ядром MVP

Вхід → пошук → складська операція → актуальний залишок → запис історії. Відхилення надмірного списання належить до того самого ядра.

3 Хто визначає пріоритети

Я визначаю продуктові пріоритети самостійно і фіксую їх у backlog. Навчальні критерії та допустимість індивідуального формату оцінює викладач.

4 Як підтверджується особистий внесок

Усі артефакти мають одного виконавця — мене. Доказами є код, звіти, результати тестів і пояснення рішень; наступні зміни варто фіксувати окремими commits.

5 Чим problem statement відрізняється від функцій

Він визначає користувача, контекст, втрату і бажану зміну. Список функцій описує спосіб реалізації цієї зміни.

6 Яка метрика доводить користь

Час знаходження потрібного товару людиною. Технічна швидкодія API допоміжна; ціль ≤30 с ще потрібно виміряти.

7 Який ризик є найбільшим

Неузгодженість залишку й історії за помилки або конкуренції. Його знижують транзакція, умовний UPDATE, обмеження БД та тести.

8 Чому .gitignore не очищує історію

Він впливає на додавання невідстежуваних файлів. Файл у старому commit залишається там; секрет, якщо такий був, потрібно відкликати, а історію очищувати окремо.